Software Requirements Specification

for

Version Control for Databases

Version 2.0

Prepared by:

Anirudh Dhoot (B24CS1026)

Ishat Varshney (B24CS1082)

Ishita Tyagi (B24CS1083)

Parrva Shah (B24CS1053)

Version 1.0 Approved by

Version Control for Databases Alpha Manager Team

comprising of Ishat Varshney and Ishita Tyagi

Date: 04/02/2026

Contents

1 Introduction **4**
1.1 Purpose . . . 4
1.2 Document Conventions . . . 4
1.3 Product Scope . . . 4

2 Overall Description **4**
2.1 Product Perspective . . . 4
2.2 Product Functions . . . 5
2.3 User Classes and Characteristics . . . 5
2.4 Operating Environment . . . 5
2.5 Design and Implementation Constraints . . . 5
2.6 User Documentation . . . 6
2.7 Assumptions and Dependencies . . . 6

3 External Interface Requirements **7**
3.1 User Interfaces . . . 7
3.2 Software Interfaces . . . 7
3.3 Communications Interfaces . . . 7

4 System Features **7**
4.1 Change Tracking . . . 7
4.1.1 Description and Priority . . . 7
4.1.2 Stimulus/Response Sequences . . . 8
4.1.3 Functional Requirements . . . 8
4.2 Version History Management . . . 8
4.2.1 Description and Priority . . . 8
4.2.2 Functional Requirements . . . 8
4.3 Rollback Mechanism . . . 8
4.3.1 Description and Priority . . . 8
4.3.2 Functional Requirements . . . 8
4.4 Snapshot Configuration Constraints . . . 9
4.5 Multi-Table Support (Future Version) . . . 9
4.5.1 Description and Priority . . . 9
4.5.2 Functional Requirements . . . 9

5 Other Nonfunctional Requirements **9**
5.1 Performance Requirements . . . 9
5.2 Security Requirements . . . 9
5.3 Software Quality Attributes . . . 9
5.4 Sprint Plan . . . 10
5.5 Project Management Tooling . . . 11

6 Design Standards and Conventions **11**
6.1 Naming Conventions 12
6.2 Code Commenting Style 12

Appendix A: Glossary **12**

Appendix B: Analysis Models **13**

Appendix C: To Be Determined List **18**

Appendix D: Requirements Traceability Matrix **18**

Revision History

Name	**Date**	**Reason for Changes**	**Version**
Project Team	21/01/2026	Initial Draft	1.0
Project Team	28/01/2026	Clarified implicit declarations	1.1
Project Team	04/02/2026	Modified Project Scope to add Workbench	2.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for the **Relational SQL Database Workbench** system. The system provides a web-based workbench for relational SQL databases that allows users to run queries through a UI and view outputs, while also providing version control capabilities for database write operations and enabling rollback to previous versions using a combination of command history, inverse operations, and mandatory snapshots.

The primary goals of this system are to:

- provide a reliable and auditable history of all successful database write operations,
- enable safe restoration of database state to a chosen earlier version,
- minimize rollback time through mandatory snapshots at user-configurable periodic intervals while balancing storage consumption, and
- preserve correctness under concurrent multi-user usage by relying on the underlying DBMS concurrency mechanisms and event ordering.
- Currently only a single table inside a database is being tracked. Project Scope will be expanded to tracking multiple tables inside a database in future versions.

1.2 Document Conventions

This document follows the IEEE Software Requirements Specification format. Sections marked as **TBD** indicate decisions that are intentionally deferred.

1.3 Product Scope

Relational SQL Database Workbench aims to provide a practical UI-based workbench for interacting with relational SQL databases. In this project, a **commit** refers to **any successful write operation** applied to the database (i.e., a successful execution of a database-modifying SQL command). The primary version of the workbench supports (1) reading from the database (e.g., SELECT) and (2) writing to the database (e.g., INSERT/UPDATE/DELETE/ALTER/DROP) via the UI, with outputs visible on the UI. The system further enables tracking of database write operations, viewing commit history, and rolling back to earlier versions with emphasis on storage efficiency and correctness.

The system focuses on maintaining a clear mapping between (1) the user-submitted write operation, (2) the resulting version identifier, and (3) the stored rollback information (snapshots and inverse operations) required to reproduce or revert database state.

2. Overall Description

2.1 Product Perspective

Version Control for Databases is a standalone system that operates alongside an existing SQL database. It intercepts data-modifying SQL commands, records them as versioned events, and maintains snapshots to enable rollback.

Major components include:

- Web-based user interface
- Backend processing service
- Event streaming layer
- Snapshot storage system

2.2 Product Functions

- Execute SQL queries via the web UI and display outputs/results on the UI
- Track SQL commands that modify database state
- Maintain a versioned commit history
- Roll back database state to previous versions
- Optimize rollback using mandatory snapshots
- Support schema-altering commands

2.3 User Classes and Characteristics

End Users: Interact through the web UI to execute commands, view history, and perform rollbacks.

System Operators: Maintain availability and storage infrastructure.

2.4 Operating Environment

- SQL-based relational database
- Backend service environment (TBD)
- Web browser
- Networked deployment environment

2.5 Design and Implementation Constraints

- Only SQL-based databases are supported
- Only data-modifying queries are tracked:
  - INSERT
  - UPDATE
  - DELETE
  - CREATE
  - ALTER
  - DROP

- SELECT queries may be executed via the workbench UI but are not tracked
- Snapshots are mandatory
- Snapshot frequency is user configurable
- Event streaming is required for concurrency handling

2.6 User Documentation

- Provide user-facing documentation to support operation of the Relational SQL Database Workbench and its version control capabilities.
- Include a short user guide describing how to submit SQL queries through the web UI, interpret displayed outputs and execution status, browse the commit (version) timeline, and initiate rollback to a selected version.
- Provide inline help that explains the version-control terms used in this specification, including commits (versions), snapshots, and inverse operations (anti-commands), and indicate where snapshot frequency can be configured.
- Provide administrative documentation describing responsibilities for maintaining system availability and snapshot storage infrastructure, and summarizing relevant configuration and environment details that are marked as TBD in this SRS.

2.7 Assumptions and Dependencies

- The system is designed for a multi-user environment where multiple users may issue database-modifying commands concurrently.
- The underlying database management system is assumed to provide native concurrency control mechanisms (e.g., locking, MVCC).
- Only databases that support built-in concurrency handling (such as PostgreSQL, MySQL, etc.) are supported in the current versions of the system.
- Manual concurrency control within the version control system is outside the scope of the current project phases.
- Support for databases lacking native concurrency mechanisms may be considered in later stages of the project.
- Event streaming infrastructure is available and used to serialize and order database-modifying events.
- Cloud-based object storage is available for snapshot persistence.

3. External Interface Requirements

3.1 User Interfaces

The system provides a web-based user interface intended to support both day-to-day database usage (read and write) and controlled rollback operations. The UI shall provide:

- an **SQL command editor** to submit SQL queries (read or write),
- an **output/results panel** that renders query results (e.g., rows/columns for SELECT) and messages for write operations (e.g., rows affected),
- a **validation and execution result view** showing success/failure status and any error messages,
- a **commit (version) timeline** that lists commits in order with metadata (timestamp, author, command summary, affected table(s)),
- a **version details view** to inspect a selected commit, including the original write command, recorded inverse operation summary, and snapshot proximity (nearest snapshot before the commit),
- a **rollback workflow** where a user selects a target version, reviews the planned rollback range (snapshot → target), confirms the action, and sees rollback progress and final status,
- a **snapshot configuration screen** to set snapshot frequency (in commits) and view recent snapshot points.

UI design specifics (layout, exact fields, and visual styling) are TBD.

3.2 Software Interfaces

- SQL database (vendor/version TBD)
- Event streaming service (implementation choice)
- Cloud object storage (implementation choice)

3.3 Communications Interfaces

- HTTP-based communication between UI and backend
- Event-based communication via streaming service

4. System Features

4.1 Change Tracking

4.1.1 Description and Priority

Tracks all database-modifying SQL commands and records them as versioned events.
Priority: High

4.1.2 Stimulus/Response Sequences

User submits SQL command → System validates → Command applied and recorded → Version updated.

4.1.3 Functional Requirements

- REQ-1: The system shall track INSERT, UPDATE, DELETE, CREATE, ALTER, and DROP commands.
- REQ-2: The system shall ignore non-modifying queries.
- REQ-3: Each tracked command shall be associated with a version identifier.

4.2 Version History Management

4.2.1 Description and Priority

The system maintains a totally ordered history of database versions derived from database-modifying operations. Each successful modification to the database is treated as a logical commit and advances the version state.
Priority: High

4.2.2 Functional Requirements

- REQ-4: The system shall treat every successful database-modifying command as a distinct commit.
- REQ-5: The system shall maintain a strictly ordered commit history using the event streaming mechanism.
- REQ-6: The commit history shall be maintained continuously from system initialization until explicitly truncated or archived.

4.3 Rollback Mechanism

4.3.1 Description and Priority

The system enables rollback to a previous database version using a combination of mandatory snapshots and inverse operations (anti-commands). Rollback operations restore the database state by first loading the nearest snapshot prior to the target version and then replaying inverse operations as required. Saga-style transactions are out of scope for this project.
Priority: High

4.3.2 Functional Requirements

- REQ-7: The system shall store inverse operations corresponding to each tracked database-modifying command.
- REQ-8: The system shall maintain mandatory snapshots of database state.

- REQ-9: During rollback, the system shall restore the nearest snapshot preceding the target version.
- REQ-10: The system shall replay inverse operations between the snapshot and the target version.
- REQ-11: If an inverse operation fails, rollback impact shall be limited to the interval between two adjacent snapshots.

4.4 Snapshot Configuration Constraints

- The system shall allow users to configure snapshot frequency in terms of number of commits.
- The default snapshot frequency shall be one snapshot per five commits.
- The maximum snapshot frequency shall be one snapshot per commit.
- The minimum snapshot frequency shall be one snapshot at system initialization, covering the entire commit history.

4.5 Multi-Table Support (Future Version)

4.5.1 Description and Priority

Extends tracking and rollback to multiple related tables.
Priority: Medium

4.5.2 Functional Requirements

- REQ-12: The system shall track dependencies between tables.
- REQ-13: Anti-commands shall account for foreign key relationships.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Latency is not a primary concern for this system.

5.2 Security Requirements

- The system shall authenticate users before allowing command execution.
- Authorization mechanisms are TBD.

5.3 Software Quality Attributes

- Storage efficiency
- Reliability
- Maintainability

5.4 Sprint Plan

The development of the Relational SQL Database Workbench is organized into three distinct sprints. The plan follows a bottom-up logic: establish a stable and testable data layer first, introduce ordering/concurrency guarantees next, and finally build the UI on top of the backend services.

- **Sprint 1: Database Setup & Core Data Operations**
  - *Goal:* Prove that the system can connect to a relational database, execute SQL reliably, and capture the required information to support later versioning.
  - *Key Deliverables:*
    * PostgreSQL instance setup and schema initialization scripts.
    * Backend functions to execute SQL reads/writes and return structured outputs (result sets, row counts, errors).
    * A minimal change-tracking data model (e.g., commit log tables / metadata) sufficient to record successful write operations.
    * Snapshot storage integration stubs (e.g., basic upload/download path to S3 or equivalent object storage).
  - *Dependencies:* None (foundation sprint).
  - *Exit Criteria (Acceptance Logic):*
    * A write query (INSERT/UPDATE/DELETE/DDL) executed through the backend returns success/failure and any output without crashing.
    * Each successful write produces a stored commit record with at least: version identifier, timestamp, and SQL text.
    * A snapshot can be created at initialization and retrieved on demand (even if not yet used for rollback).
  - *Logic:* Rollback and concurrency cannot be validated until the project can consistently run SQL operations, capture outputs, and persist minimal metadata and snapshots.
- **Sprint 2: Concurrency & Versioning/Rollback Core**
  - *Goal:* Introduce deterministic ordering of write operations and implement rollback using snapshots and inverse operations.
  - *Key Deliverables:*
    * Event ordering logic (via an event stream / queue) so concurrent writes are serialized into a strict commit history.
    * Generation and persistence of inverse operations (anti-commands) for supported write operations.
    * Rollback engine that: (1) selects the nearest snapshot, (2) restores it, and (3) applies inverse operations to reach the target version.
    * Snapshot frequency configuration and a mechanism to create snapshots periodically.

- *Dependencies:* Stable SQL execution + commit metadata + snapshot persistence from Sprint 1.
- *Exit Criteria (Acceptance Logic):*
  - Under concurrent requests, commits are assigned monotonically increasing version identifiers in a single total order.
  - For a chosen target version, rollback completes successfully in the common case and reports failure conditions clearly.
  - After rollback, read queries (SELECT) reflect the restored state.
- *Logic:* Once operations are reliable, the next risk is correctness under concurrency and correctness of rollback semantics.

- **Sprint 3: Frontend Integration & UI Workbench**
  - *Goal:* Deliver an end-to-end usable workbench where users can run SQL, see outputs, and initiate rollback.
  - *Key Deliverables:*
    - Web UI with SQL editor and output/results panel for both reads and writes.
    - UI views for commit history (version timeline) and version details.
    - Rollback UI flow (select version → confirm → show progress/result).
  - *Dependencies:* Stable backend APIs for execute/query, history retrieval, and rollback (from Sprints 1–2).
  - *Exit Criteria (Acceptance Logic):*
    - A user can run SELECT queries and see tabular results in the UI.
    - A user can run write queries and see success status and updated commit timeline.
    - A user can select a prior version and initiate rollback, then verify the new state via SELECT.
  - *Logic:* The UI is a consumer of the backend services; building it after core correctness reduces rework and makes integration predictable.

5.5 Project Management Tooling

The team utilizes **Trello** for Agile project management.

- **Rationale:** Trello's Kanban-style boards provide high visual clarity for tracking the lifecycle of specific features (from TODO to Sprint-specific backlogs).
- *Usage:* Each Sprint is assigned a dedicated list. Cards include specific technical requirements, such as "UML diagrams update" or "Identify alpha managers," ensuring that documentation and administrative tasks progress alongside code.

6. Design Standards and Conventions

To ensure code consistency and maintainability across the Android application, backend service, and database layers, the following standards are mandated.

6.1 Naming Conventions

The project adopts a strict naming policy to minimize cognitive load during cross-layer development:

Element	**Naming Style**	**Standard**	**Example**
Classes	PascalCase	Nouns, singular	`DatabaseAdapter`, `RollbackManager`
Methods	camelCase	Verb-Noun pair	`executeWrite()`, `loadSnapshot()`
Variables	camelCase	Descriptive nouns	`versionId`, `sqlCommand`
Constants	UPPER_CASE	Fixed values	`DEFAULT_SNAPSHOT_FREQ`
Database Tables	lower_case	Plural nouns	`commit_events`, `snapshots`

6.2 Code Commenting Style

The following commenting standards are used across the codebase:

- Use GitHub-like keywords such as `TODO` and `FIXME` to mark planned work and known issues.
- When addressing another team member in a comment, mention their name/identifier explicitly.
- When adding a non-trivial comment, include the comment author (name/identifier) so ownership is clear.

Appendix A: Glossary

- **Version**: Logical state of the database after applying a set of commands
- **Snapshot**: Stored representation of database state at a version
- **Anti-command**: Inverse operation used for rollback

Appendix B: Analysis Models

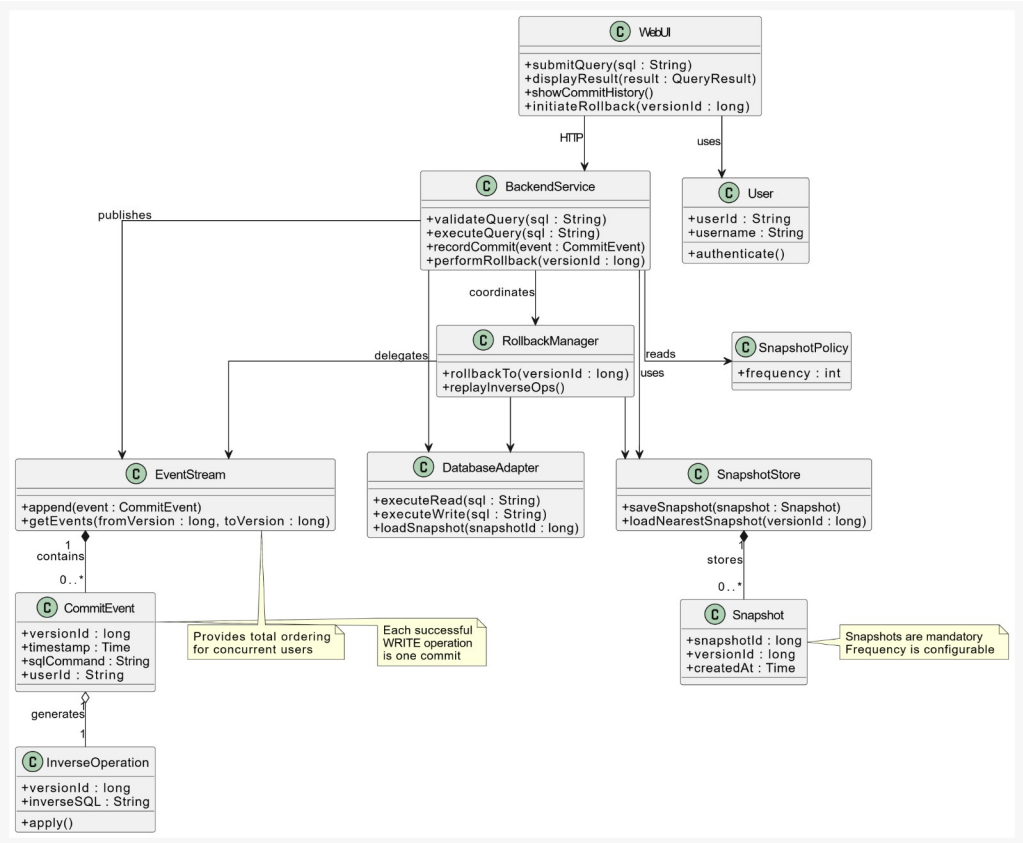

Figure 1: Class Diagram

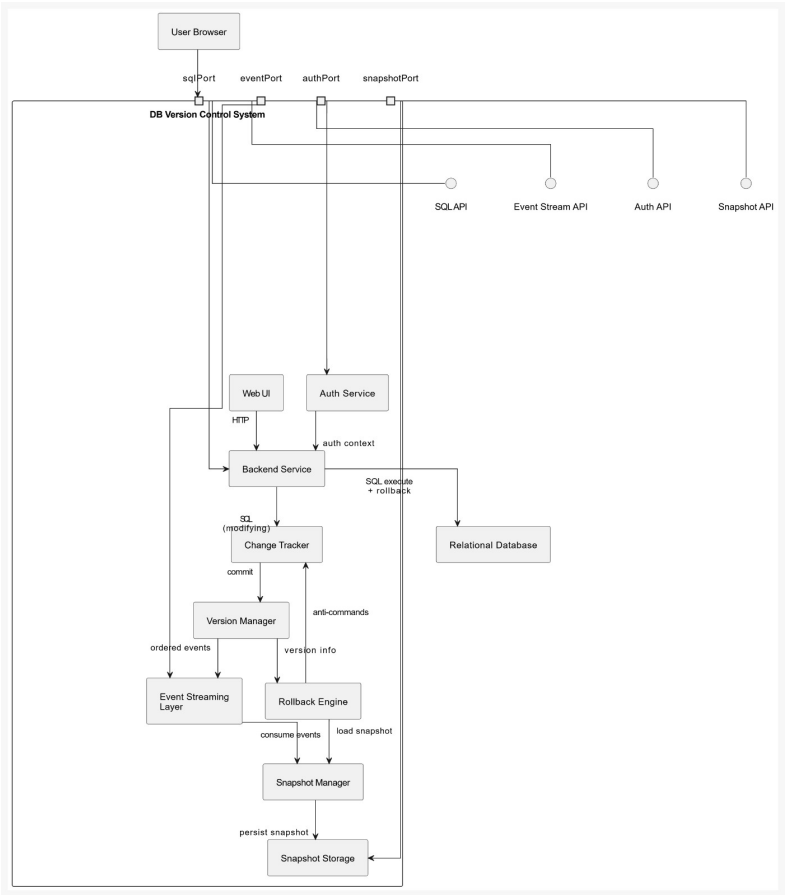

Figure 2: Composite Structure Diagram

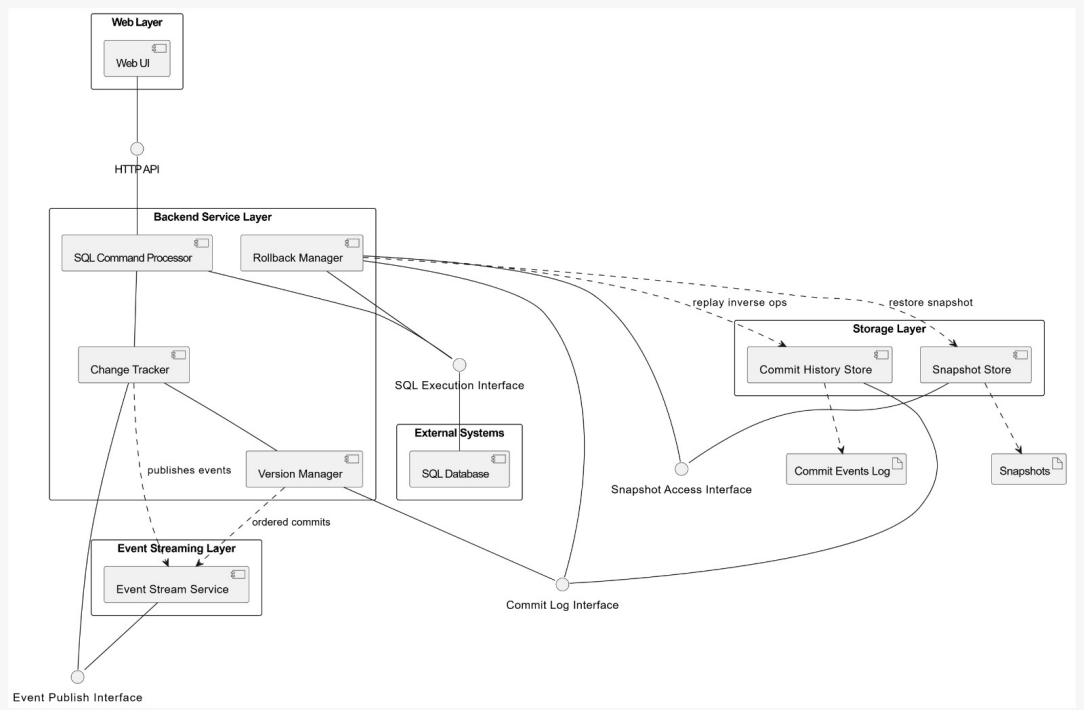

Figure 3: Component Diagram

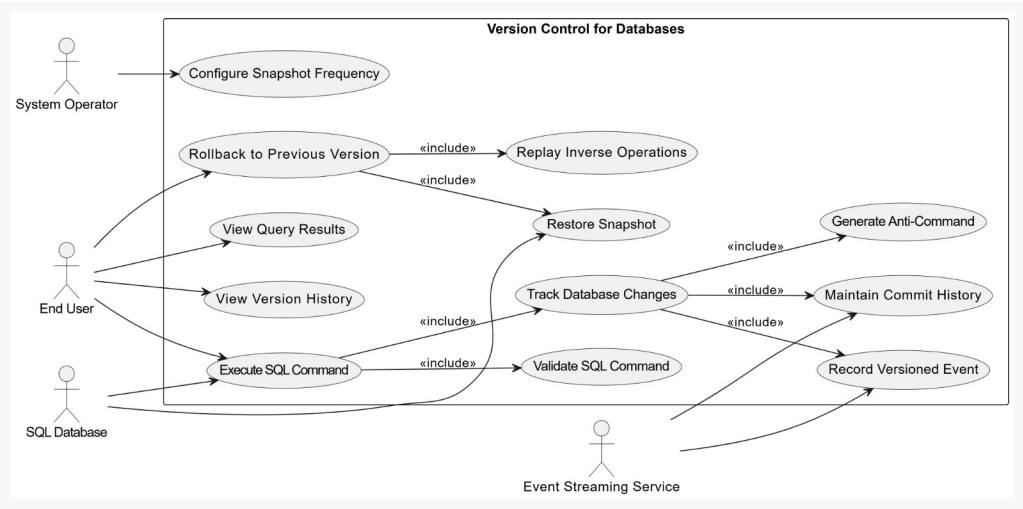

Figure 4: Use Case Diagram

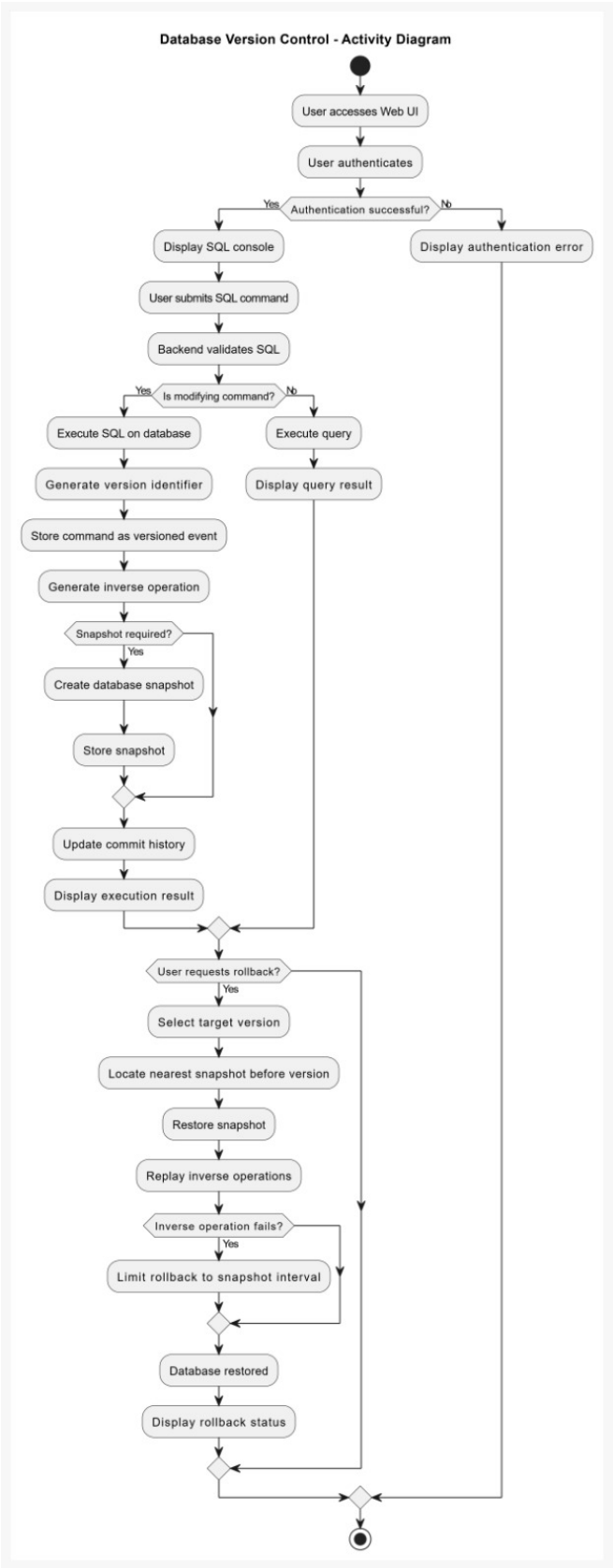

Figure 5: Activity Diagram

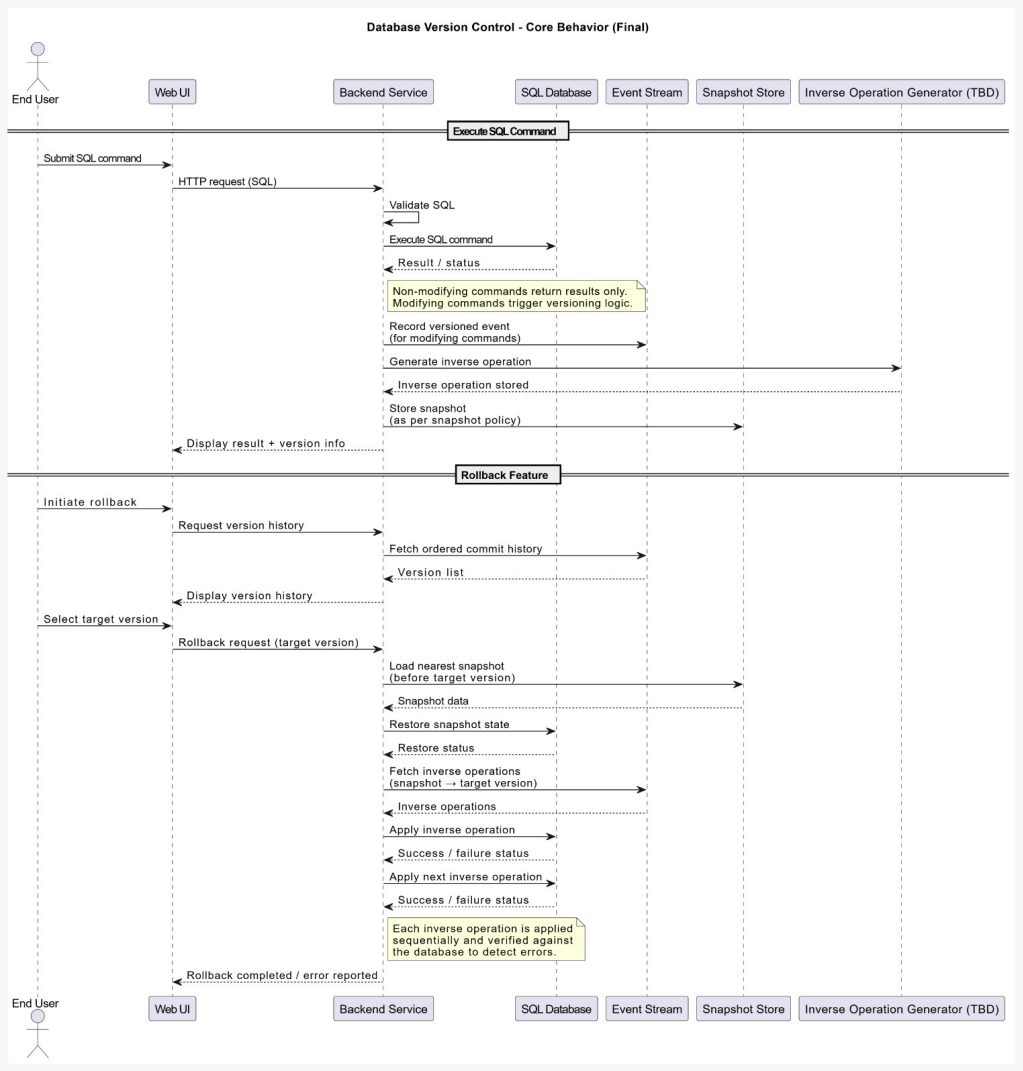

Figure 6: Sequence Diagram

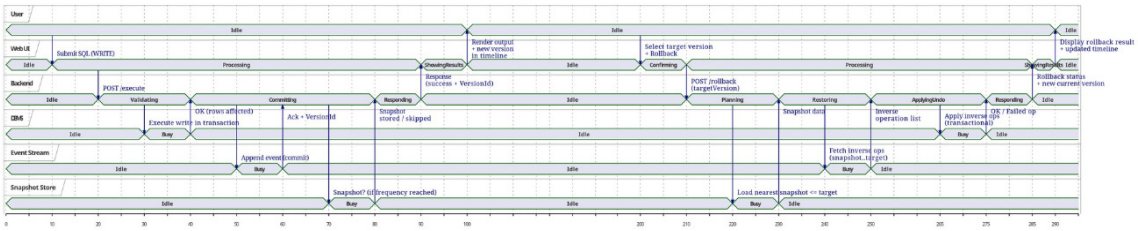

Figure 7: Timing Diagram

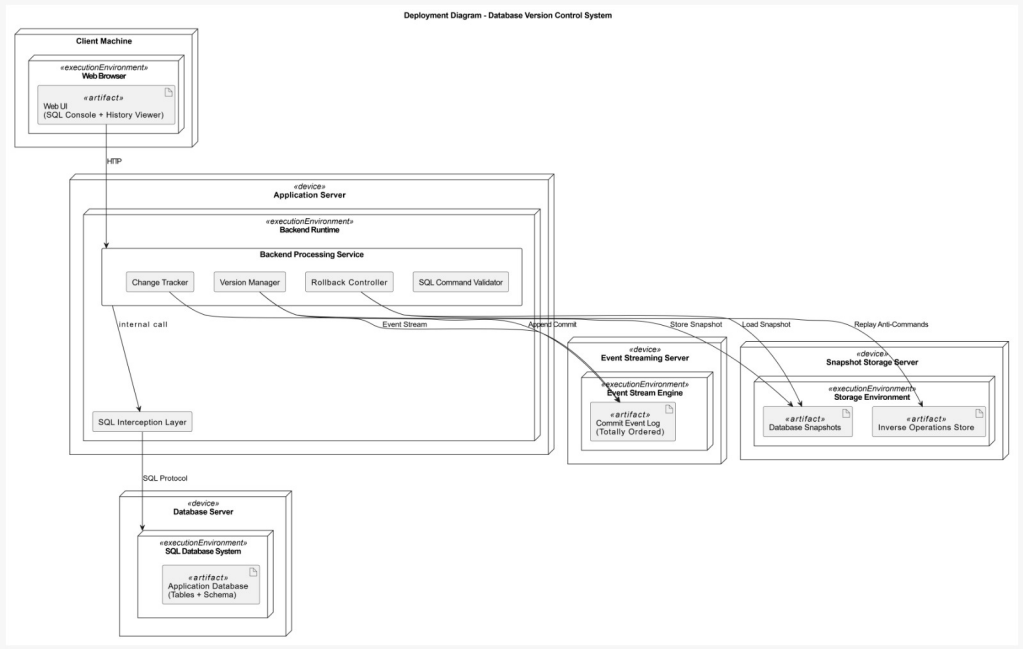

Figure 8: Deployment Diagram

Appendix C: To Be Determined List

1. Snapshot frequency
2. Database vendor/version
3. UI design details
4. Authorization rules
5. Deployment environment

Appendix D: Requirements Traceability Matrix

Attribute	Requirements (Functional / Non-functional / Business)	Component	Class	Functions	Start Date of Code	End Date of Code	Update Date	Update Made	Reason for Update	Team Member(s) Designing Code	Team Member(s) Updating Code
Description	REQ-1: Track INSERT, UPDATE, DELETE, CREATE, ALTER, DROP commands	Change Tracker	BackendService	validateQuery(), executeQuery(), recordCommit()							
	REQ-2: Ignore non-modifying SQL queries	Change Tracker	BackendService	validateQuery()							
	REQ-3: Associate each tracked command with a version identifier	Version Manager	Commit	Commit(versionId, sqlCommand, timestamp)							
	REQ-4: Treat every successful modifying command as a distinct commit	Version Manager	EventStream	appendEvent(commit)							
	REQ-5: Maintain strictly ordered commit history using event streaming	Event Streaming Layer	EventStream	getEvents(fromVersion, toVersion)							
	REQ-6: Maintain continuous commit history	Storage Layer	CommitHistoryStore	storeCommit(), fetchCommits()							
	REQ-7: Store inverse operations for each tracked command	Rollback Engine	InverseOperation	apply()							
	REQ-8: Maintain mandatory snapshots of database state	Snapshot Manager	SnapshotStore	saveSnapshot(), loadNearestSnapshot()							
	REQ-9: Restore nearest snapshot preceding target version	Rollback Engine	RollbackManager	rollbackToVersion(versionId)							
	REQ-10: Replay inverse operations between snapshot and target version	Rollback Engine	RollbackManager	replayInverseOps()							
	REQ-13: Limit rollback impact to interval between adjacent snapshots	Rollback Engine	RollbackManager	rollbackToVersion(versionId)							

Table 3: Requirements Traceability Matrix (Current State)